

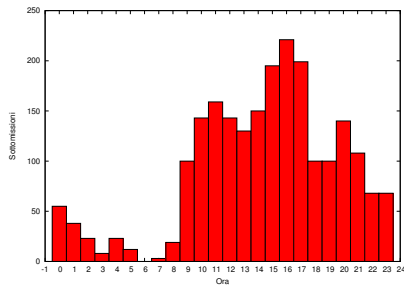
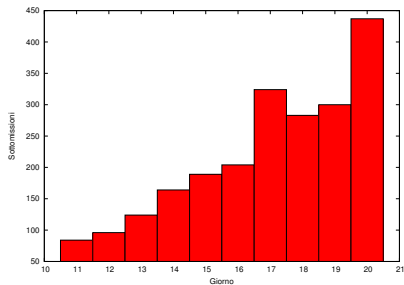
Spiegazioni ASD 2012

Luddisti Spaziali

Risultati

Statistiche

Numero sottoposizioni: 2088



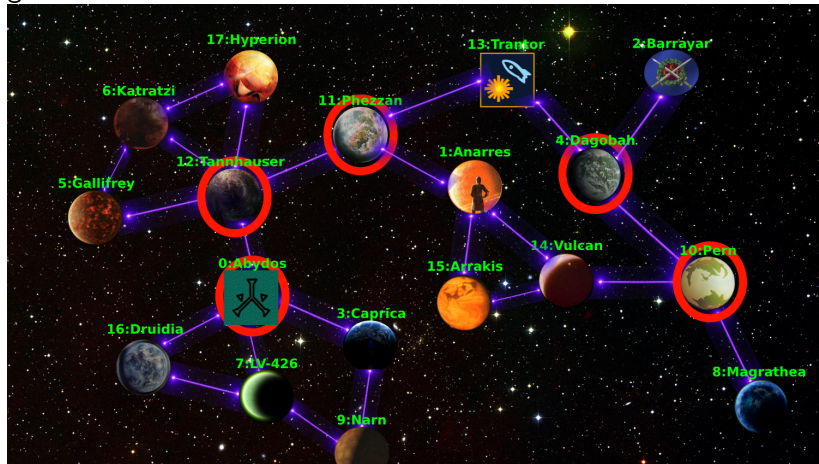
Punteggi

- ▶ $P < 30 \rightarrow$ progetto non passato
- ▶ $30 \leq P < 75 \rightarrow$ un punto bonus al voto dello scritto
- ▶ $75 \leq P \rightarrow$ due punti bonus al voto dello scritto

Classifica completa su judge, nella pagina delle slides.

Punti di articolazione

Un nodo é un punto di articolazione se la sua rimozione rende il grafo non connesso.



Algoritmo $O(V(V + E))$

Controllo per ogni nodo se la sua rimozione ha effetti.

```
for all Nodes  $n \in G.N$  do  
    for all Nodes  $m \in G.N$  do  
         $m.visited = false$   
    end for  
     $n.visited = true$   
     $s = n.vicini[0]$   
     $num = visita(G, s)$   
    if  $num \neq (|G.N| - 1)$  then  
         $n.articulation = true$   
    end if  
end for
```

Componenti rimanenti

Rimuoviamo ogni punto di articolazione ed analizziamo le componenti del grafo.



Serve un viceré in ogni componente verde, nessuno nelle componenti blu.

Ragionamento

Le componenti che hanno almeno due punti di articolazione come vicini non hanno bisogno di un viceré.



Correttezza

Lemma

Le componenti con un solo punto di articolazione come vicino hanno bisogno di un viceré.

Dimostrazione

Se i luddisti attaccano quel punto di articolazione, la componente risulta sconnessa dal resto del grafo.

Lemma

Le componenti con almeno due punti di articolazione come vicini non hanno bisogno di un viceré.

Dimostrazione

Intuitivamente, possiamo attraversare il punto di articolazione non attaccato per muoversi in una componente vicina. Se questa ha solo un punto di articolazione, avrà un viceré. Se ha più di un punto di articolazione, utilizziamo quello che non abbiamo attraversato per raggiungere un'altra componente. Visto che non ci possono essere cicli, prima o poi raggiungeremo una componente con un singolo viceré.

Algoritmo per il posizionamento $O(N + V)$

```
function POSIZIONA
  for all Nodes  $n \in G.N$  do
    if  $n.articulation = true$  then
       $n.visited = true$ 
    else
       $n.visited = false$ 
    end if
  end for
  for all Nodes  $n \in G.N$  do
    if  $n.visited = false$  then
       $check(n)$ 
    end if
  end for
end function
```

```
function CHECK(Node  $st$ )
  Stack  $s$ 
  Set  $articulation$ 
   $s.push(st)$ 
  while  $!s.empty()$  do
     $n = s.top(); s.pop()$ 
    if  $n.visited = false$  then
       $n.visited = true$ 
      for all Node  $m \in n.vicini$  do
         $s.push(m)$ 
      end for
    else
      if  $n.articulations = true$  then
         $articulations.insert(n)$ 
      end if
    end if
  end while
  if  $articulations.size \leq 1$  then
     $n.vicere = true$ 
  end if
end function
```

Recap

Algoritmo

- ▶ Trovare punti di articolazione ($O(V(V + E))$)
- ▶ Posizionare i vicere ($O(V + E)$)

Complessità: $O(V(V + E))$

Circa 75 punti su judge

Algoritmo ottimo

É possibile trovare i punti di articolazione in $O(V + E)$

Classificazione degli archi

La dfs costruisce un albero T .

Lemma

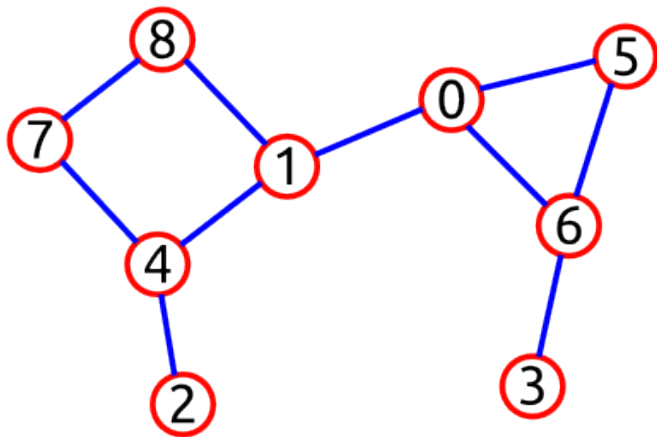
Sia $(u, v) \in (E \setminus T)$ un arco non incluso in T , allora u è un antenato di v o viceversa. Non ci sono archi di attraversamento.

Dimostrazione

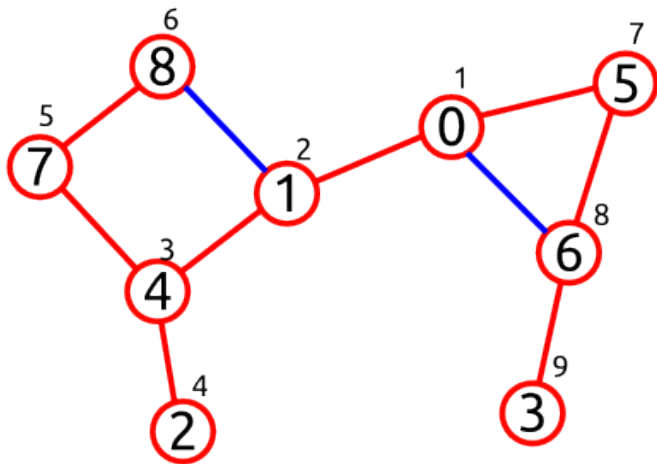
Supponiamo di analizzare (u, v) durante la visita di u . I casi sono i seguenti:

- ▶ È la prima volta che incontriamo v : v diventa figlio di u in T .
- ▶ La visita di v è in corso: u è un discendente di v
- ▶ La visita di v è terminata: impossibile. u è un vicino di v e quindi avrebbe dovuto essere visitato prima che v finisse la sua visita.

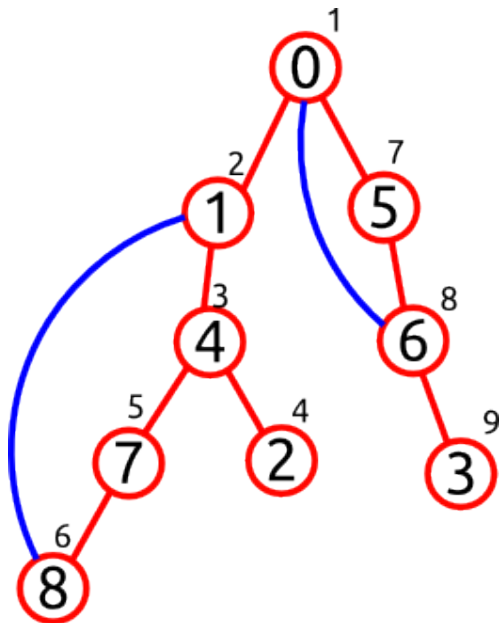
Esempio di grafo non orientato



Esempio di visita partendo da 0



Albero dfs partendo da 0

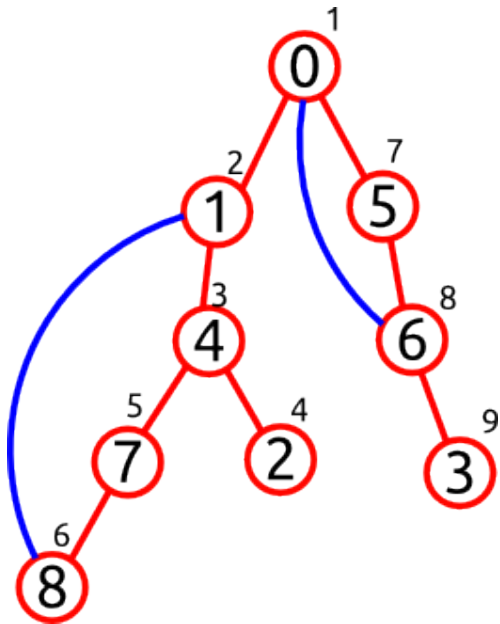


Un nodo N é un punto di articolazione se il sottoalbero di uno dei suoi figli non ha archi verso un predecessore di N .

Per ogni nodo manteniamo:

- ▶ Il suo livello nell'albero (level)
- ▶ Il livello piú alto che riesce a raggiungere dai nodi del suo sottoalbero (low)

Albero dfs partendo da 0



Id	Level	Low
0	0	0
1	1	0
2	3	2
3	3	2
4	2	1
5	1	0
6	2	0
7	3	1
8	4	1

Caso speciale: La radice è un punto di articolazione se ha almeno due figli nell'albero dfs.

Algoritmo

Visita iniziata da nodo s

```
function DFS(Node  $n$ )  
   $n.low = n.level$   
  for all Node  $v \in n.vicini$  do  
    if  $v.level = -1$  then  
       $v.level = n.level + 1$   
       $dfs(v)$   
       $n.low = \min(n.low, v.low)$   
      if ( $n \neq s$  and  $v.low \geq n.level$ ) or ( $n = s$  and  $v \neq n.vicini[0]$ ) then  
         $n.articulation = true$   
      end if  
    else  
       $n.low = \min(n.low, v.level)$   
    end if  
  end for  
end function
```

Conclusioni

Soluzioni

Sulla pagina delle slides su Judge ci sono le soluzioni “ufficiali” e tutte le vostre soluzioni.

Secondo Progetto

- ▶ Da 1 a 3 punti bonus
- ▶ Dal 16 al 25 Maggio